

# An Integrated Perception, Navigation, and Manipulation System for Autonomous Robotic Tasks

Robotic Navigation and Exploration – Final Project Technical Report

An-Yi Lee   Student ID: S11259029

Graduate-level External Coursework   June 2026

## Abstract

This individual final project develops an autonomous mobile manipulation system in the PROS Twin simulation environment by integrating visual perception, localization, navigation, task-level control, and robotic-arm manipulation. The system uses YOLO-based object detection together with depth information to identify task-relevant objects, AMCL and Nav2 for map-based localization and navigation, and a finite-state task controller to coordinate perception-driven behaviors.

Three task pipelines were implemented: object retrieval and return, object transport across a bridge, and door-handle interaction. In the final demonstration, the system successfully completed the first two tasks. For the third task, the perception, visual alignment, and manipulation procedures were implemented, but the complete door-opening task was not successfully completed. The project therefore focuses not only on individual modules, but also on the practical challenges of integrating perception, navigation, and manipulation into a multi-stage autonomous robotic system.

## 1 Introduction

Autonomous robots rarely complete real tasks with a single perception or control component. A typical indoor manipulation task may require the robot to identify a target in the camera view, estimate its relative position, move through a mapped environment, perform precise alignment near the target, and finally coordinate a robotic arm to interact with it. Although each module can be tested independently, reliable execution depends heavily on how information and control are transferred between modules.

This final project for the *Robotic Navigation and Exploration* course focuses on that integration problem. The system is implemented in the PROS Twin simulation environment using ROS 2. It combines YOLO-based visual perception, RGB-D depth sensing, AMCL localization, Nav2 navigation, direct vehicle commands, and robotic-arm control. A task-level finite-state controller coordinates these components and selects different motion strategies according to the current stage of the task. This project was completed individually. I was responsible for integrating the perception, localization, navigation, vehicle-control, and robotic-arm components, as well as designing the task-control logic, implementing task-specific behaviors, debugging the system, and conducting the final demonstration.

The project contains three increasingly complex task pipelines. **Task 1** requires the robot to search for a bear, align with it, grasp it, and return it to the starting region. **Task 2** extends the retrieval behavior by requiring the robot to carry the bear across a bridge before returning to the designated region. **Task 3** requires the robot to approach a door region, detect the door handle, align the vehicle to the handle, and perform a handle-pressing and door-pushing sequence. Together, these tasks cover object perception, global navigation, constrained traversal, close-range alignment, and manipulation.

The main engineering challenge is that the most suitable controller changes across task stages. Nav2 is effective for global movement in mapped open space, but it does not provide the precision required for grasping or door-handle interaction. Conversely, purely local image-based control is useful near a target but inefficient for long-distance travel. The final system therefore adopts a hybrid strategy:

global map-based navigation is used when appropriate, local visual feedback is used for precise alignment, and empirically tuned motion sequences are used in constrained regions such as the bridge and door approach.

## 2 Project Environment and Task Requirements

### 2.1 PROS Twin and ROS 2 Environment

The project is executed in PROS Twin, which provides the simulated mobile platform, robotic arm, RGB-D camera, and environment required for the final tasks. ROS 2 is used as the communication layer between perception, navigation, localization, vehicle control, and arm control. Foxglove is used during development to inspect the map, robot pose, camera streams, and perception outputs simultaneously.

The final project was assessed primarily through a live demonstration rather than a fixed offline benchmark. Consequently, the report focuses on the implemented task pipeline, observed end-to-end behavior, and engineering decisions required to obtain stable execution.

### 2.2 Task Definition

**Table 1:** Summary of the final-project tasks.

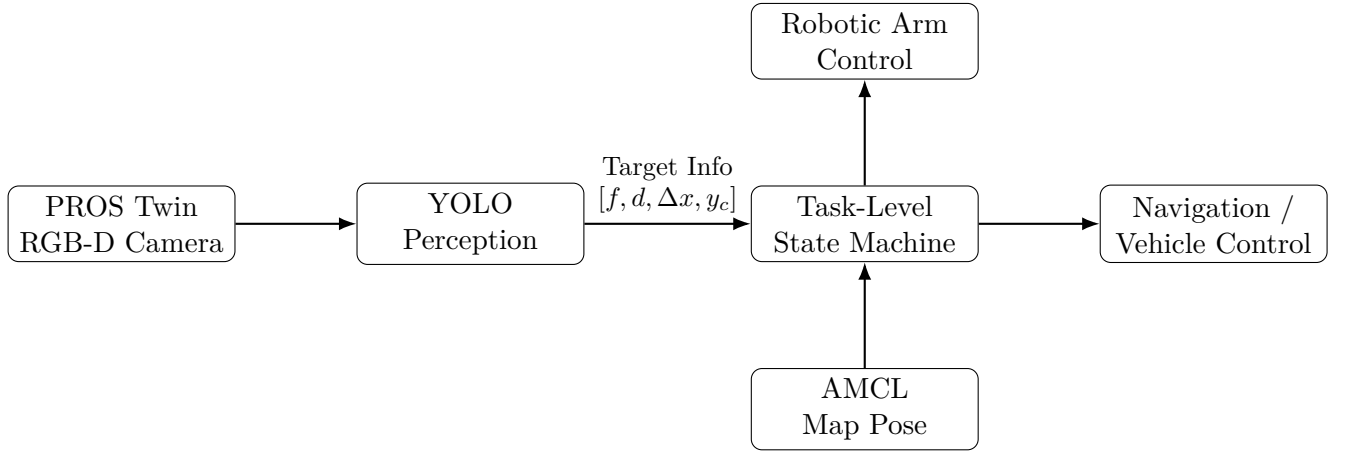
| Task   | Goal                                                              | Required Capabilities                                                                                                  |
|--------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Task 1 | Retrieve a bear and return it to the starting region.             | Object detection, depth estimation, visual alignment, grasping, AMCL/Nav2 return navigation, object release.           |
| Task 2 | Carry a bear across a bridge and return to the designated region. | Object retrieval, stable grasp, constrained bridge traversal, recovery after leaving the bridge, return navigation.    |
| Task 3 | Interact with a door handle and open the door.                    | Door-handle detection, coarse approach, fine visual alignment, arm positioning, handle pressing, forward door pushing. |

## 3 System Architecture

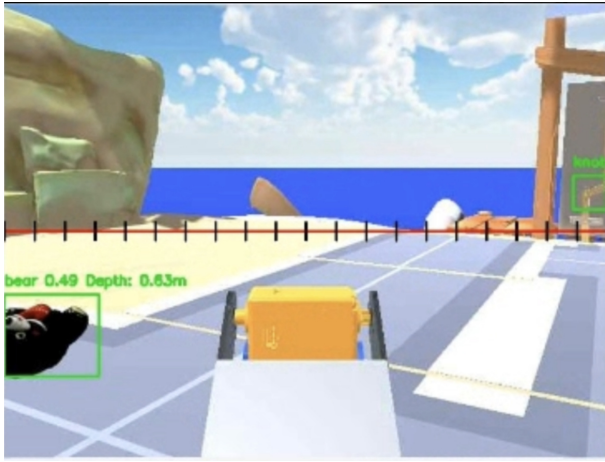
Figure 1 illustrates the final system. The RGB image is processed by the YOLO perception node while the depth stream is used to estimate the distance associated with the detected target. The perception module publishes a compact target message containing whether a target is found, its distance, its horizontal offset from the image center, and the vertical position of its bounding box center. In parallel, AMCL provides the current robot pose in the map frame. The task controller combines these observations and decides whether to continue searching, issue local vehicle commands, request Nav2-based movement, or trigger arm manipulation.

### 3.1 Visual Perception

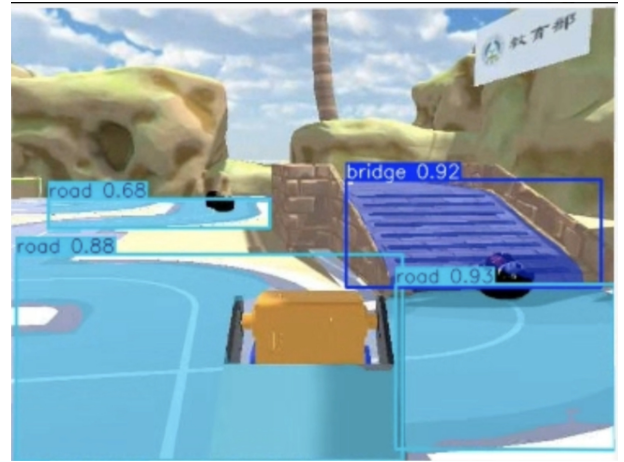
The perception component was developed from the earlier object-detection and semantic-segmentation homework in the same environment. A custom dataset of 82 images was collected from PROS Twin and labeled in Roboflow. For object detection, the target classes were **bear** and **knob**; a YOLO11n detector was trained for 50 epochs at an image size of 640 with batch size 8. A separate YOLO11n-seg model for road and bridge segmentation was also developed during the course, although the final task controller primarily relies on bear/knob detection and depth information.



**Figure 1:** High-level architecture and control flow of the final robotic system.



(a) Bear/knob object detection.



(b) Road/bridge semantic segmentation.

**Figure 2:** Perception models developed in the PROS Twin project environment.

For each selected target, the final controller receives

$$\mathbf{o} = [f, d, \Delta x, y_c], \quad (1)$$

where  $f$  denotes target availability,  $d$  is the estimated depth in meters,  $\Delta x$  is the horizontal displacement of the target from the image center, and  $y_c$  is the vertical center of the bounding box. The horizontal offset is used for steering corrections, while  $d$  and  $y_c$  together are used to judge whether the robot is sufficiently close for manipulation.

### 3.2 Localization, Navigation, and Task Control

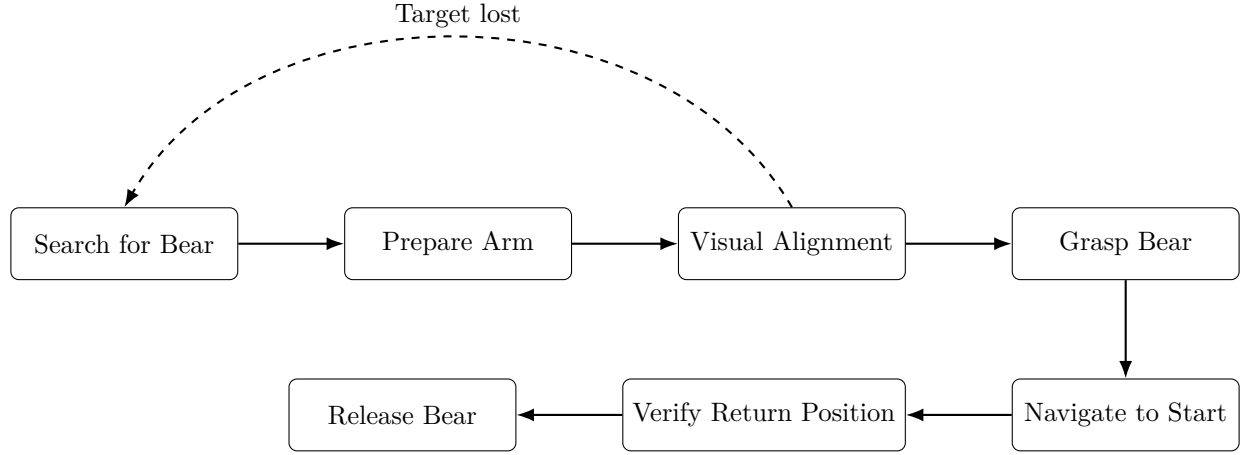
AMCL supplies the current map-frame pose  $(x_r, y_r, \theta_r)$ . Nav2 is used for coarse movement and return navigation. During task execution, the starting AMCL position is recorded dynamically instead of relying only on a fixed home coordinate. This allows the robot to return to the actual starting region after completing an object-manipulation sequence.

The task-level controller is implemented as a finite-state machine. Each state performs one limited behavior—for example searching, final alignment, arm preparation, grasping, navigation, or release—and transitions only when its corresponding conditions are satisfied. This design makes failures easier to isolate and allows perception-specific recovery behaviors to be inserted without rewriting the entire task pipeline.

## 4 Task Implementation

### 4.1 Task 1: Bear Retrieval and Return

Task 1 is the core perception-guided manipulation pipeline. Its simplified sequence is shown in Fig. 3.



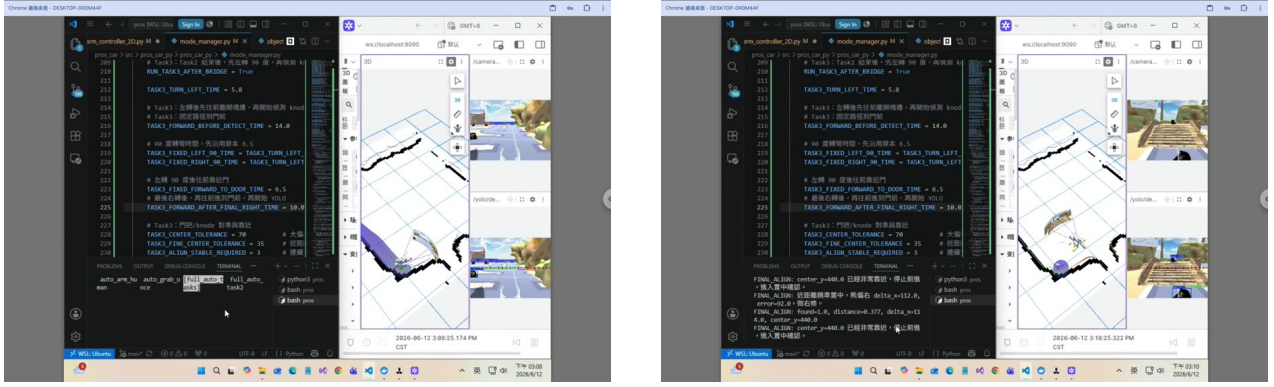
**Figure 3:** Task 1 execution flow. If the target is lost during visual alignment, the controller returns to the search stage.

**Stable target acquisition.** The robot does not lock onto the bear after a single detection. The search state first checks whether the object is visible, roughly centered, and within a plausible distance range. Multiple consistent observations are required before the system proceeds. If the bear is not visible, the robot performs short rotational search actions. Repeated loss triggers a return to the broader search behavior instead of allowing the controller to remain stuck in a close-range state.

**Close-range visual alignment.** After the target is acquired, the arm is moved into a safer pre-grasp posture and the robot enters a fine-alignment state. The vehicle makes small left/right corrections according to  $\Delta x$  and advances in short steps when the bear is near the image center. At very close range, the controller requires several consecutive centered observations before grasping. This reduces oscillation and prevents a transient detection from immediately triggering the arm.

**Depth and image-position fusion.** Near the bear, raw depth can occasionally be inconsistent. The controller therefore does not use depth alone. The vertical image position  $y_c$  is treated as an additional closeness cue: when the bear occupies the lower portion of the image, forward motion is stopped even if the reported depth remains unexpectedly large. The approach condition consequently combines geometric distance with an image-space heuristic.

**Grasp and return.** Once alignment is stable, a fixed reachable point in the arm base frame is used to initiate the grasping routine. After the bear is secured, the robot returns to the stored starting position using Nav2. A navigation finish flag is treated only as a hint; the state machine transitions to release only when the AMCL distance to the stored goal is within a tolerance. This additional check was introduced after early versions sometimes released the bear before the robot had physically returned to the intended region.



(a) Bear visible in the camera and YOLO view.

(b) Close-range alignment during the integrated demo.

**Figure 4:** Examples from the recorded development demonstration. The terminal logs expose the active task state while Foxglove shows localization and perception.

## 4.2 Task 2: Carrying the Bear Across a Bridge

Task 2 extends the retrieval pipeline by requiring the robot to preserve the grasp while traversing a narrow bridge. The task first places the robot near the bridge-front region, reuses the bear search and final-alignment logic, and then enters a dedicated carrying state once the bear is grasped.

The bridge segment is handled differently from open-space navigation. Rather than allowing the global planner to repeatedly reorient the robot while it is carrying the bear, the final implementation uses an empirically tuned local motion sequence to maintain a consistent heading and sufficient forward momentum across the bridge. The arm remains in its grasping configuration during traversal. After leaving the bridge, the robot performs a short recovery maneuver to clear the constrained area and then switches back to Nav2 for the longer return movement.

This hybrid strategy combines Nav2 for long-range movement with task-specific local control for predictable traversal of the narrow bridge segment.

## 4.3 Task 3: Door-Handle Interaction

Task 3 introduces a different type of manipulation target. The door handle is substantially smaller than the bear and is useful only after the robot reaches the correct side of the door. The implementation therefore separates *coarse approach* from *fine visual interaction*.

The robot first follows a tuned coarse path that moves it from the bridge/starting region to the door area. YOLO-based handle detection is activated as the final alignment signal only after this approach, reducing the chance that distant or irrelevant detections affect the task. Once a handle is detected, the controller checks confidence, horizontal offset, depth, and whether the bounding-box center lies within a plausible vertical range for a door handle. Several centered observations are required before the robot is considered aligned.

If the handle is centered but still too far away, the robot advances in small increments and re-runs perception after every step. When the handle is sufficiently close, the arm is reset to a controlled posture and moved toward a predefined point near the handle. Instead of executing the bear grasp routine, the arm performs a downward pressing motion. The planned sequence then commands the vehicle to move forward while the arm remains engaged in order to push the door.

Although the complete perception, alignment, and manipulation pipeline was implemented, the full door-opening task was not successfully completed during the final on-site demonstration. The robot was able to detect the door handle and perform the designed alignment procedure, but the interaction between close-range perception, vehicle positioning, and robotic-arm manipulation remained insufficiently robust for consistent task completion. Compared with the previous two tasks, this task required substantially higher positioning accuracy and tighter coordination between

perception and manipulation.

## 5 Robustness Mechanisms and Design Decisions

The final behavior was obtained through repeated debugging of interactions between perception, navigation, and manipulation. The following mechanisms were particularly important.

**Multi-frame confirmation.** A single detection can be noisy or momentary. Major state transitions therefore require repeated stable observations, especially before close-range grasping or handle interaction.

**Target-loss recovery.** During search and final alignment, loss of the target triggers controlled rotational search. If the target remains missing for too long, the state machine returns to an earlier search state instead of continuing an unsafe approach.

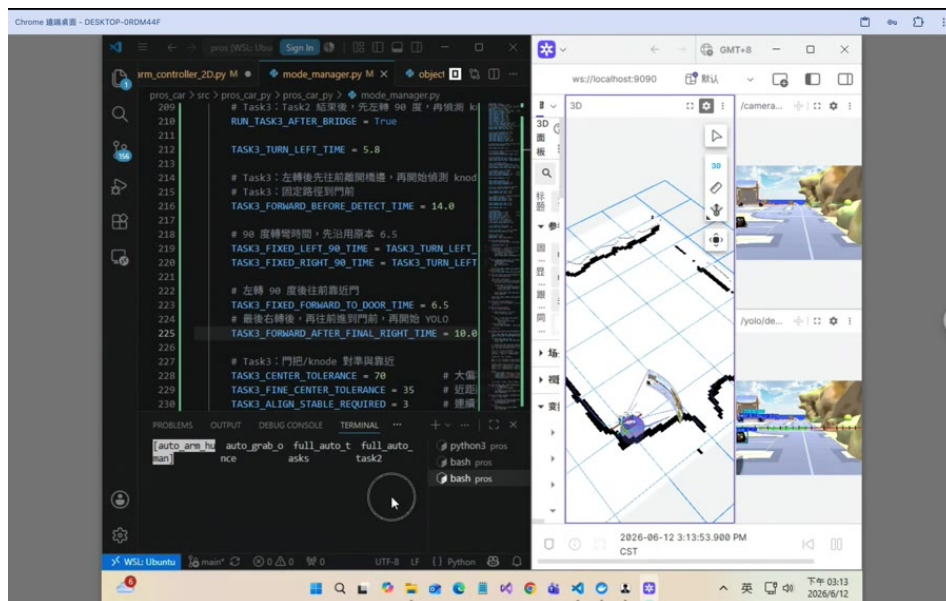
**Depth fallback through image geometry.** Depth is useful but not always reliable near objects. The target’s vertical image position is therefore used as an additional cue to prevent unnecessary forward motion when the target is already visually very close.

**Dynamic home pose.** The starting AMCL pose is recorded during task initialization. Return navigation can therefore target the actual starting region rather than depending only on a hand-entered map coordinate.

**AMCL-based return verification.** The starting AMCL pose is recorded during task initialization so that the robot can return to the actual starting region. In addition, Nav2’s finish signal is not used as the sole completion criterion; before object release, the controller verifies that the current AMCL position is within a specified distance of the recorded return goal.

## 6 Demonstration and Results

The final project was evaluated through a live end-to-end demonstration in the PROS Twin environment. During development, Foxglove was used to monitor the 3D map, localized robot pose, raw camera stream, and YOLO output, while terminal logs displayed task-state transitions and control decisions. Figure 5 shows the integrated debugging and demonstration environment.



**Figure 5:** Integrated demonstration view with source code and controller logs on the left and Foxglove localization, camera, and perception views on the right.

Table 2 summarizes the final on-site demonstration results. Tasks 1 and 2 were successfully completed,

while Task 3 was partially completed.

**Table 2:** Summary of the final demonstration results.

| Task   | Demonstrated Capabilities                                                     | Status              |
|--------|-------------------------------------------------------------------------------|---------------------|
| Task 1 | Bear detection, visual alignment, grasping, Nav2 return, and object release   | Completed           |
| Task 2 | Bear transport, bridge traversal, post-bridge recovery, and return navigation | Completed           |
| Task 3 | Door-handle detection, close-range alignment, and robotic-arm interaction     | Partially Completed |

Tasks 1 and 2 demonstrate that perception, navigation, vehicle control, and manipulation can be coordinated within a single autonomous task pipeline. Task 3 implemented the perception and alignment stages as well as the planned manipulation sequence; however, the complete door-opening procedure was not successfully completed during the final demonstration.

Because the course final project was evaluated through a live demonstration rather than repeated randomized trials, no statistical success-rate benchmark was conducted. The results should therefore be interpreted as an end-to-end system demonstration rather than a statistically controlled robotics evaluation.

## 7 Conclusion

This individual final project developed an autonomous robotic system integrating visual perception, localization, navigation, task-level control, and robotic-arm manipulation in the PROS Twin environment. YOLO and depth information were used for target perception and local alignment, while AMCL and Nav2 supported map-based localization and global navigation. A finite-state controller coordinated these components throughout the multi-stage tasks.

In the final demonstration, the system successfully completed the bear retrieval and return task and the bear transport and bridge traversal task. For the door-interaction task, handle detection, visual alignment, and the planned manipulation procedure were implemented, but the complete door-opening task was not successfully completed. Overall, the project demonstrated the practical integration of perception, global navigation, local control, and manipulation, while also exposing limitations caused by perception uncertainty, positioning accuracy, and task-specific control strategies.